FYEO

Spree

Repo: https://github.com/Spree-Finance/Spreev2

Security Review Update: March 10, 2026

Reviewer: reviewer@gofyeo.com

Security Code Review of Spree v2

New security issues, 10

After the development team implemented the latest updates, FYEO conducted a review of the modifications. The primary goal of this evaluation was to ensure the continued robustness of the program's security features, safeguarding user funds and maintaining the overall integrity of the program.

General Updates:

The latest update introduces a comprehensive conditional rewards system centered around the new PendingSPVault contract, which manages “pending SP” tokens that are backed 1:1 by actual SP tokens held in escrow. This system enables campaign-scoped reward distribution where each campaign has its own budget, minting limits, and whitelisted destinations for both settlement and transfers. Users can earn pSP tokens through various campaigns, and these tokens can either be settled into real SP tokens when conditions are met, or expired and returned to the campaign budget if conditions aren’t satisfied. The update also adds tier-based reward boosting through integration with a BoostEngine, allowing users with higher tier status to receive multiplied rewards automatically when minting pSP tokens. Additionally, an EscrowManager contract provides programmable release conditions for pSP tokens, supporting time-locked, milestone-based, two-party approval, and arbitrated escrow types for more complex reward distribution scenarios.

The update also includes significant improvements to the existing vault infrastructure, particularly around security and gas optimization. BonusRewardsVault now enforces mandatory token balance verification on harvest operations to prevent accounting manipulation, adds a maximum TWAB share cap to protect against oracle attacks, and includes a dust sweeping function for recovering excess tokens. The TWABTracker contract has been completely refactored with period-based checkpoint collation that overwrites updates within the same hour rather than appending new checkpoints, dramatically reducing gas costs while maintaining accurate time-weighted balance tracking. SpreeVault4626 received enhanced reentrancy protection in its TWAB update flow and now performs authorization checks before making external calls to avoid wasted gas. Several smaller fixes address initialization issues in the Aave adapter, add fee receiver validation, and introduce helper contracts like SpreeIntegration for partners to easily gate features based on user tier levels.

EscrowManager ignores campaignId causing multi-campaign users to be locked out

Finding ID: FYEO-SPREE-01
Severity: **High**
Status: **Remediated**

Description

All escrow creation functions accept a `campaignId` parameter but completely ignore it when transferring pSP tokens. This causes a critical incompatibility: users with pSP from multiple campaigns cannot create escrows at all, even when specifying which campaign to use.

The root cause is that `_transferPspIn` ignores the `campaignId` parameter and uses standard ERC20 `transferFrom`, which PendingSPVault only allows when the user has exactly one campaign. When a user has multiple campaigns, PendingSPVault reverts with `CampaignIdRequired()`.

Proof of Issue

File name: src/managers/EscrowManager.sol
Line number: 709

```
function _transferPspIn(address from, uint256 amount, bytes32 campaignId)
internal {
    // Transfer pSP from user to this contract
    // User must have approved this contract on PendingSPVault
    pspToken.safeTransferFrom(from, address(this), amount);  // campaignId
ignored
}

function _transferPspOut(address to, uint256 amount, bytes32 campaignId)
internal {
    // Transfer pSP from this contract to recipient
    // Note: This uses standard ERC20 transfer. For campaign-scoped transfers,
    // the EscrowManager would need TRANSFER_ADMIN_ROLE on PendingSPVault
    pspToken.safeTransfer(to, amount);  // campaignId ignored
}
```

File name: src/PendingSPVault.sol
Line number: 675

```
function transferFrom(address from, address to, uint256 amount) public override
returns (bool) {
    // Only allow if admin and user has single campaign
    if (hasRole(MANAGER_ROLE, msg.sender) || hasRole(TRANSFER_ADMIN_ROLE,
msg.sender)) {
        bytes32[] memory campaigns = getUserCampaigns(from);
        if (campaigns.length == 1) {  // ONLY works with single campaign
            // ... transfer logic
        }
    }
    revert CampaignIdRequired();  // Reverts for multi-campaign users
}
```

Scenario:
1. User receives pSP from campaign A 2. User receives pSP from campaign B 3. User tries to create escrow: `createTimeLocked(campaignA, beneficiary, 500, ...)` 4. EscrowManager calls `pspToken.safeTransferFrom(user, escrowManager, 500)` 5. PendingSPVault checks `getUserCampaigns(user).length`, returns 2 6. PendingSPVault reverts with `CampaignIdRequired()` 7. User cannot create any escrow, despite having sufficient balance and specifying the campaign

Severity and Impact Summary

This breaks core protocol functionality for any user participating in multiple campaigns. The escrow system becomes unusable for these users, effectively locking their pSP tokens.

Additional impact: The campaignId parameter in EscrowManager is completely ignored during transfers, allowing users to create escrows with mismatched campaign associations. For example, a user with pSP from Campaign A can successfully create an escrow claiming it's for Campaign B (which they don't have pSP for), causing the escrow record to show Campaign B while the actual funds are deducted from Campaign A. This breaks campaign budget tracking and accounting integrity, making it impossible to reliably audit which campaigns are funding which escrows, and could lead to incorrect financial reporting.

Recommendation

Replace standard ERC20 transfers with campaign-aware transfers.

Grace Period Can Be Set to Zero, Disabling Protection

Finding ID: FYEO-SPREE-02
Severity: **Low**
Status: **Remediated**

Description

The `setGracePeriodDuration` function allows setting grace period to 0, which would effectively disable grace period protection for users. While this might be intentional, it could lead to immediate tier demotions without warning, especially if changed during active user sessions.

Proof of Issue

File name: src/tiers/SpreeStatusRegistry.sol
Line number: 256

```
function setGracePeriodDuration(bytes32 namespace, uint256 duration)
    external
    validNamespace(namespace)
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (duration > MAX_GRACE_PERIOD) revert InvalidGracePeriod();
    _gracePeriods[namespace] = duration;
    emit GracePeriodDurationSet(namespace, duration);
}
```

Severity and Impact Summary

Setting grace period to 0 could cause unexpected immediate demotions for users who are currently in grace period, potentially affecting their rewards and user experience negatively.

Recommendation

Add a minimum grace period requirement.

Oracle Can Set Unconfigured or Out-of-Bounds Tiers

Finding ID: FYEO-SPREE-03
Severity: **Low**
Status: **Remediated**

Description

The `setTierByOracle` function allows an oracle to set any tier for a user in partner namespaces without validating that the tier has been properly configured with a valid `TierConfig`.

While the function has basic validation, it lacks sanity checks against the namespace's actual tier configuration. This means an oracle could set a user to tier 4 even if the namespace only configured tiers 0-2, or assign a tier that was never configured.

Proof of Issue

File name: src/tiers/SpreeStatusRegistry.sol
Line number: 225

```
function setTierByOracle(address user, bytes32 namespace, uint8 tier, uint64
score)
    external
    validNamespace(namespace)
    validTier(tier)  // Only checks tier <= MAX_TIER_LEVEL (4)
    whenNotPaused
    nonReentrant
    onlyRole(ORACLE_ROLE)
{
    ...
    _setTier(user, namespace, tier, score);
}
```

Severity and Impact Summary

While the oracle is trusted, the lack of sanity checks means: - Compromised oracle could grant invalid tier levels - Oracle bugs could assign unconfigured tiers - Users could receive multipliers from unconfigured tiers (defaults to MIN_MULTIPLIER) - Breaks namespace isolation and admin control over tier structure

Recommendation

Add validation in `setTierByOracle` to ensure tier assignments respect namespace configuration.

TWAB Query Failure Causes Revert in Batch Operations

Finding ID: FYEO-SPREE-04
Severity: **Low**
Status: **Remediated**

Description

In `_getTWAB`, any failure in the TWAB query causes a revert with `TWABQueryFailed()`. While this prevents silent demotions, it also means that in `batchUpdateTiers`, a single user's TWAB query failure will cause the entire batch to revert, preventing updates for all other users in the batch.

Proof of Issue

File name: src/tiers/SpreeStatusRegistry.sol
Line number: 473

```
(bool success, bytes memory data) = twabTracker.staticcall(
    abi.encodeWithSignature("getTWAB(address,address,uint256,uint256)",
spToken, user, startTime, endTime)
);

if (!success || data.length < 32) revert TWABQueryFailed();

return abi.decode(data, (uint256));
```

Severity and Impact Summary

A single malformed user address or TWAB tracker issue can block tier updates for all users in a batch, potentially causing widespread tier calculation delays and affecting reward distribution.

Recommendation

Consider implementing try-catch logic in batch operations to skip problematic users while continuing to process others, or emit an event for failed updates rather than reverting the entire batch.

Arbitrated escrow depositor can release without arbiter

Finding ID: FYEO-SPREE-05
Severity: **Informational**
Status: **Acknowledged**

Description

In `Arbitrated` escrows, the depositor can call `release()` without arbiter approval, defeating the purpose of arbitration.

Proof of Issue

File name: src/managers/EscrowManager.sol
Line number: 756

```
} else if (escrow.escrowType == EscrowType.Arbitrated) {
    // Only depositor can release (or manager)
    if (msg.sender != escrow.depositor && !hasRole(MANAGER_ROLE, msg.sender)) {
        revert NotDepositor();
    }
}
```

Severity and Impact Summary

The depositor can release funds unilaterally. The arbiter only becomes relevant after `raiseDispute()` is called.
Depositor can front-run `raiseDispute()` by calling `release()` first. “Arbitrated” implies arbiter control, but they’re just a passive dispute resolver

Current design makes “Arbitrated” functionally similar to “TimeLocked” (depositor-controlled) with optional dispute resolution, not true arbitrated escrow.

Recommendation

Make sure this is implemented as intended. Explicitly state arbiter has no release approval authority.

Integer overflow in milestone loop counter

Finding ID: FYEO-SPREE-06
Severity: **Informational**
Status: **Remediated**

Description

In the `createMilestone` function, the loop counter is declared as `uint8` which can only hold values 0-255. If `MAX_MILESTONES` is ever increased beyond 255 or if the validation is bypassed, the loop counter will overflow silently.

Proof of Issue

File name: src/managers/EscrowManager.sol
Line number: 275, 301, 517

```
for (uint8 i = 0; i < milestones.length; i++) {
    if (milestones[i].amount == 0) revert InvalidAmount();
    totalAmount += milestones[i].amount;
}
```

Severity and Impact Summary

While `MAX_MILESTONES` is currently 20, using `uint8` creates a potential footgun if the constant is ever changed. The validation at line 264 (`if (milestones.length > MAX_MILESTONES) revert TooManyMilestones();`) prevents the overflow in practice.

If `MAX_MILESTONES` is increased beyond 255 in a future upgrade without updating the loop counter type, the increment `i++` would revert with an overflow panic at i=255. This would make the `createMilestone` function permanently unusable for any milestone array with >255 elements.

Recommendation

Use `uint256` for loop counters consistently throughout the contract, or add a compile-time assertion that `MAX_MILESTONES <= 255`.

Missing Maximum Bound for boostMultiplier in TierConfig

Finding ID: FYEO-SPREE-07
Severity: **Informational**
Status: **Remediated**

Description

The `TierConfig` struct's `boostMultiplier` parameter lacks a maximum bound validation. While the implementation validates a minimum (`MIN_MULTIPLIER = 100`) and `holdingPeriod` has proper bounds (`MIN_HOLDING_PERIOD = 1 days`, `MAX_HOLDING_PERIOD = 365 days`), there is no maximum limit on `boostMultiplier`. This could allow administrators to accidentally set unrealistic multipliers (e.g., 1,000,000 = 10,000x) that would break the tier system's economic model.

Proof of Issue

File name: src/tiers/SpreeStatusRegistry.sol
Line number: 333

```
if (config.boostMultiplier < MIN_MULTIPLIER) {
    revert InvalidMultiplier();
}
// No maximum bound check
```

Severity and Impact Summary

Without a maximum bound, administrators could accidentally set multipliers like a 10,000x boost, which would: - Break the economic model of reward distribution - Potentially drain reward pools - Create unfair advantages - Make the system economically unsustainable

Recommendation

Add a `MAX_MULTIPLIER` constant and validate against it in `setTierConfig`.

TWABHelper Library Usage Risks

Finding ID: FYEO-SPREE-08
Severity: **Informational**
Status: **Acknowledged**

Description

The `TWABHelper` library is not used in this codebase but may be used by external repositories. The library has two potential concerns:

1. **Silent failures**: `safeUpdateTotalSupply` and `safeUpdateBalance` return `false` on failure instead of reverting. If calling contracts don't check return values, TWAB tracking could silently fail while token balances update, causing state inconsistency.
2. **Gas-intensive authorization**: `canUpdate()` makes three sequential external calls (`address check`, `twabEnabled`, `authorizedUpdaters`) with try-catch blocks for every balance update, which could be expensive in high-frequency scenarios.

Proof of Issue

File name: src/libraries/TWABHelper.sol
Line number: canUpdate, safeUpdateTotalSupply, safeUpdateBalance

```
function canUpdate(ITWABTracker tracker, address token) internal view returns
(bool) {
    // Three external calls with try-catch
}

function safeUpdateTotalSupply(...) internal returns (bool success) {
    // Returns false on failure, doesn't revert
}
```

Severity and Impact Summary

The library is unused in this codebase. If used externally, risks depend on implementation.

Recommendation

For external repositories using this library: ensure all return values are checked and consider emitting events on failures. The gas concern is minor.

TWABTracker uint80 balance limit

Finding ID: FYEO-SPREE-09
Severity: **Informational**
Status: **Remediated**

Description

TWABTracker uses `uint80` for balance storage, limiting trackable balances to ~1.2M tokens (18 decimals). Vaults exceeding this limit will revert on TWAB updates.

Proof of Issue

File name: src/TWABTracker.sol
Line number: 167, 199

```
if (newBalance > type(uint80).max) revert BalanceOverflow();
```

Severity and Impact Summary

The team has confirmed this limit is intentional for their current use case. However, this constraint should be documented for future deployments or if vault scale changes.

Recommendation

Document the 1.2M token limit in deployment documentation and consider adding validation during vault setup to prevent enabling TWAB for vaults that may exceed this threshold.

Two-party escrow mutual approval requirement

Finding ID: FYEO-SPREE-10
Severity: **Informational**
Status: **Acknowledged**

Description

In `TwoParty` escrows, both depositor and beneficiary must approve for release. When both `autoReleaseAfter` and `autoRefundAfter` are set to 0, the escrow requires mutual approval with no automatic timeout mechanism. This design requires trust between parties or manager intervention if disputes arise.

Proof of Issue

File name: src/managers/EscrowManager.sol
Line number: 326

```
function createTwoParty(
    bytes32 campaignId,
    address beneficiary,
    uint256 amount,
    uint64 autoReleaseAfter,
    uint64 autoRefundAfter
) external nonReentrant whenNotPaused returns (bytes32 escrowId) {
    // ...
    // At least one auto-action must be set, or both disabled
    if (autoReleaseAfter != 0 && autoRefundAfter != 0) {
        if (autoReleaseAfter >= autoRefundAfter) revert InvalidDuration();
    }
```

Severity and Impact Summary

The team has confirmed that mutual approval is the intended behavior. This design choice allows for flexible escrow configurations where parties can opt for manual resolution. However, users should be aware that without timeout settings, escrows depend on both parties cooperating or manager intervention.

Recommendation

Update documentation to clarify that both-zero timeout configuration is intentional and requires mutual trust or manager oversight for dispute resolution.

Commit Hash Reference:

For transparency and reference, the security review was conducted on the specific commit hash for the Spree repository. The commit hash for the reviewed versions is as follows:

4fc350e5c63ba73a102142fafc667ba212a4bbc6

Remediations have been submitted with the commit hash 12453d77009f15ab99f2988382a72e13f8da4446.

Conclusion:

In conclusion, the security aspects of the Spree program remain robust and unaffected by the recent updates. Users can confidently interact with the protocol, assured that their funds are well-protected. The commitment to security exhibited by the development team is commendable, and we appreciate the ongoing efforts to prioritize the safeguarding of user assets.